

Initialization [Home](#)

These are the functions:

Function	Meaning
<u>lcInitialize</u>	Performs LiteCAD system initialization
<u>lcUninitialize</u>	Performs LiteCAD system uninitialization

If your application will work with camera device using **IC Imaging Control DLL** then you also have to call the following functions :

Function	Meaning
<u>lcTIS_InitLibrary</u>	Attaches tisgrabber.dll
<u>lcTIS_CloseLibrary</u>	Detaches tisgrabber.dll

Access Objects Properties [Home](#)

LiteCAD objects property can be one of the following types:

- Boolean (true or false),
- Integer (32 or 64 bit, depends on Windows bits),
- Float (64-bit double float),
- String (Unicode, 16 bit per character),
- Handle (32 or 64 bit pointer, depends on Windows bits)

There are a set of functions to read/write objects properties, depend on the type:

Function	Meaning
<u>lcPropGetBool</u>	Returns a property's value of boolean type
<u>lcPropGetInt</u>	Returns a property's value of integer type
<u>lcPropGetFloat</u>	Returns a property's value of floating type
<u>lcPropGetStr</u>	Returns a property's value of string type
<u>lcPropGetStrA</u>	Used to get a property's value of ANSI string type (for VB6)
<u>lcPropGetHandle</u>	Returns a property's value of handle type (pointer)
<u>lcPropPutBool</u>	Sets a property's value of boolean type
<u>lcPropPutInt</u>	Sets a property's value of integer type
<u>lcPropPutFloat</u>	Sets a property's value of floating type
<u>lcPropPutStr</u>	Sets a property's value of string type
<u>lcPropPutHandle</u>	Sets a property's value of handle type (pointer)

Each property has a symbolic name, written with template

LC_PROP_*object***_***propname*, where *object* - object name, and the *propname* is a property name, for example:

LC_PROP_LAYER_NAME - layer name

it has type "String", therefore, in order to set layer name you have to call:

lcPropPutStr (hLayer, LC_PROP_LAYER_NAME, szName);

Some properties have more then one type, for example, you can set active layer either by its name or by handle:

lcPropPutStr (hDrw, LC_PROP_DRW_LAYER, L"Streets");

lcPropPutHandle (hDrw, LC_PROP_DRW_LAYER, hLayer);

Code sample:

```
bool bRulers;
int Size;
double X, Y;
WCHAR szFileName[256];
COLORREF Color;

// read properties
bRulers = lcPropGetBool( hLcWnd, LC_PROP_WND_RULERS );
wcscpy( szFileName, lcPropGetStr( hImage, LC_PROP_IMAGE_FILENAME ) );
Size = lcPropGetInt( 0, LC_PROP_G_GRIPSIZE );
```

LiteCAD API reference

```
Color = lcPropGetInt( 0, LC_PROP_G_GRIPCOLOR );
X = lcPropGetFloat( hLcWnd, LC_PROP_WND_CURSORX );
Y = lcPropGetFloat( hLcWnd, LC_PROP_WND_CURSORX );

// write properties
lcPropPutInt( hLcWnd, LC_PROP_WND_COLORBG, RGB(0,0,0) );
lcPropPutInt( hLcWnd, LC_PROP_WND_COLORCURSOR, RGB(255,255,255) );
lcPropPutBool( 0, LC_PROP_G_SAVECFG, true );
lcPropPutInt( hLcWnd, LC_PROP_WND_LWMODE, LC_LW_PIXEL );
lcPropPutFloat( hLcWnd, LC_PROP_WND_LWSCALE, 0.2 );
lcPropPutStr( 0, LC_PROP_G_DLGVAl, szFileName );
```

Global properties [Home](#)

LiteCAD system variables are global properties, therefore in the lcProp... functions use NULL for the hObject parameter.

Global properties can be set/get before calling of lcInitialize function.

Property	Type	Access	Meaning
LC_PROP_G_REGCODE	string	R	Registration code
LC_PROP_G_VERSION	string	R	LiteCAD version
LC_PROP_G_MSGTITLE	string	RW	title for messages (default "LiteCAD")
LC_PROP_G_HELPFILE	string	RW	Name of help file ("Litecad.chm" by default)
LC_PROP_G_DIRDLL	string	R	Directory of running Litecad.dll module
LC_PROP_G_DIRFONTS	string	RW	Directory of font files (*.lcf), by default <LC_PROP_G_DIRDLL>\Fonts
LC_PROP_G_DIRLNG	string	RW	Directory of language files (*.lng), by default <LC_PROP_G_DIRDLL>\Config
LC_PROP_G_DIRTPL	string	RW	Directory of drawing's templates (*.lcd), by default <LC_PROP_G_DIRDLL>\Templates
LC_PROP_G_DIRCFG	string	RW	Directory of config files (the directory must have write permit), by default <LC_PROP_G_DIRDLL>\Config
LC_PROP_G_SAVECFG	bool	RW	Determines whether to save configuration or not
LC_PROP_G_ICON16	handle string	RW W	Handle to icon for LiteCAD dialogs (size 16x16) Icon filename
LC_PROP_G_ICON32	handle string	RW W	Handle to icon for LiteCAD dialogs (size 32x32) Icon filename
LC_PROP_G_RULERBMP	handle string	W	Handle to bitmap for rulers corner Filename of raster image
LC_PROP_G_PRNUSEBMP	bool	RW	Use bitmap for printing
LC_PROP_G_PRNBMPFILE	string	RW	File to save the print bitmap
LC_PROP_G_PRNBOTNRAS	bool	RW	Visibility of button "Raster..." in the "Print" dialog
LC_PROP_G_PRNBOTNSRIF	bool	RW	Visibility of items "Save Raster Image to File" in the "Print/Raster"

Property	Type	Access	Meaning
LC_PROP_G_PNTPIXSIZE	bool	RW	dialog Meaning of negative PointSize parameter, if TRUE - size in pixels, FALSE - % of window height
LC_PROP_G_GETDELENT	bool	RW	Controls if lcBlockGetEnt... functions will retrieve deleted entities (LC_PROP_ENT_DELETED) or not
LC_PROP_G_SBARHEIGHT	int	R	<u>StatusBar</u> default height
LC_PROP_G_FILEPROGRESS	bool	RW	Display progress box during file load/save
LC_PROP_G_FILEDEFEXT	string	RW	Default file extension used for "Open file" dialog
LC_PROP_G_FILELCD	boolg	RW	Enable "*.lcd" file filter in Open/Save dialogs
LC_PROP_G_ORDSEQ_AUTO	bool	RW	Auto-sort connected entities (<u>command</u> LC_CMD_ORDER_SEQ)
LC_PROP_G_TABCMDWND	bool	RW	Display command window by pressing the <Tab> key
LC_PROP_G_UNDOMSG	bool	RW	Display a message when Undo/Redo buffer is empty
LC_PROP_G_MINCHARSIZE	int	RW	Ranges 3..15. Below this value a text character will be drawn as a rectangle
LC_PROP_G_PANREDQUAL	bool	RW	Reduce draw quality when panning
LC_PROP_G_ENTEXT	bool	RW	Display entity's extents rectangle
LC_PROP_G_DLGVAL	string	RW	Dialog value for <u>lcDgGetValue</u> function
LC_PROP_G_STR	string	RW	Text value
LC_PROP_G_INT	int	R	Integer value
LC_PROP_G_FLOAT	float	R	Float value
LC_PROP_G_HANDLE	handle	R	Handle value
LC_PROP_G_DELKEYERASE	bool	RW	Using <Delete> key for "Erase" command (LC_CMD_ERASE)
LC_PROP_G_DEMOTEXT	string	RW	Demo text drawn over a window
LC_PROP_G_DEMOSIZE	int	RW	Height of demo text (pixels)
LC_PROP_G_DEMOCOLOR	int	RW	Color of demo text (COLORREF)
<u>Selection of entities</u>			
LC_PROP_SEL_PICKBOXSIZE	int	RW	Half-size of selecting square, in pixels
LC_PROP_SEL_PICKBYRECT	bool	RW	Implied windowing for objects selection (AutoCAD PICKAUTO)

Property	Type	Access	Meaning
LC_PROP_SEL_PICKDRAG	bool	RW	Selection Rect: Press and Drag (same as AutoCAD PICKDRAG system variable)
LC_PROP_SEL_PICKADD	bool	RW	Controls whether subsequent selections replace the current selection set or add to
LC_PROP_SEL_PICKBYLAYER	bool	RW	Select entities only on active layer
LC_PROP_SEL_PICKINPGON	bool	RW	Select polygons by click on inner area
LC_PROP_SEL_PICKINPGONF	bool	RW	LC_PROP_SEL_PICKINPGON works only for filled polygons
LC_PROP_SEL_PICKINIMG	bool	RW	Select images by click on inner area
LC_PROP_SEL_COLORL1	int	RW	Line color of selected entities (COLORREF)
LC_PROP_SEL_COLORL2	int	RW	Line's gap color of selected entities (COLORREF)
LC_PROP_SEL_COLORF	int	RW	Fill color of selected entities (COLORREF)
LC_PROP_SEL_HATCHFILL	bool	RW	Draw hatch filling when entity is selected
LC_PROP_SEL_ENABLEGRIPS	bool	RW	Display grips of selected entities
LC_PROP_SEL_GRIPSIZE	int	RW	Half-size of grip square, in pixels
LC_PROP_SEL_GRIPCOLORF	int	RW	Grips filling color (COLORREF)
LC_PROP_SEL_GRIPCOLORB	int	RW	Grips border color (COLORREF)
LC_PROP_SEL_GRIPENTLIM	int	RW	Max number of selected entities to display grips
LC_PROP_SEL_GRIPNUM	bool	RW	Display grips numbers (for polyline)
<u>Object Snap</u>			
LC_PROP_G_OSNAP_MARK	bool	RW	Display object snap marker
LC_PROP_G_OSNAP_APER	bool	RW	Display object snap aperture box
LC_PROP_G_OSNAP_MARKSIZE	int	RW	Half-size of marker indicating snap point, pixels (1-15)
LC_PROP_G_OSNAP_APERSIZE	int	RW	Half-size of square around cursor, used for snap, pixels (1-15)
LC_PROP_G_OSNAP_MARKCOLOR	int	RW	Color of object snap marker (COLORREF)
<u>Aerial View</u>			
LC_PROP_G_NAV_LEFT	int	RW	Left of "Aerial view" window
LC_PROP_G_NAV_TOP	int	RW	Top of "Aerial View" window
LC_PROP_G_NAV_WIDTH	int	RW	Width of "Aerial View" window

Property	Type	Access	Meaning
LC_PROP_G_NAV_HEIGHT	int	RW	Height of "Aerial View" window
<u>Jump Lines</u>			
LC_PROP_G_JL_EALEN	int	RW	Length of entity's arrow (pixels)
LC_PROP_G_JL_EAW	int	RW	Half-width of entity's arrow (pixels)
LC_PROP_G_JL_JALEN	int	RW	Length of jump arrow (pixels)
LC_PROP_G_JL_JAW	int	RW	Half-width of jump arrow (pixels)
LC_PROP_G_JL_ACOLOR	int	RW	Color of arrows, RGB value (COLORREF)
LC_PROP_G_JL_NUMFONT	string	RW	Numbers font name, empty string means Windows default GUI font
LC_PROP_G_JL_NUMSIZE	int	RW	Numbers font size, ignored if default GUI font is used
LC_PROP_G_JL_NUMCOLOR	int	RW	Color of numbers, RGB value (COLORREF)
LC_PROP_G_JL_DRAWJARR	bool	RW	Draw arrows of jump lines (between entities)
LC_PROP_G_JL_DRAWJLINE	bool	RW	Draw jump lines (between entities)
LC_PROP_G_JL_DRAWEDOT	bool	RW	Draw dots on entity lines
LC_PROP_G_JL_DRAWEARR	bool	RW	Draw arrows on entity lines
LC_PROP_G_JL_DRAWENUM	bool	RW	Draw numbers on entity lines
<u>Camera view</u>			
LC_PROP_G_CAMERA_COUNT	int	RW	Number of camera devices
LC_PROP_G_CAMERA_I	int	RW	Set camera index
LC_PROP_G_CAMERA_NAME	string	R	Name of camera by index (LC_PROP_G_CAMERA_I)
LC_PROP_G_CAMERA_ON	bool	R	TRUE if camera is connected
LC_PROP_G_CAMERA_TIME	int	RW	Interval between camera shots (msec)
LC_PROP_G_CAMERA_WIDTH	int	R	Width of camera image (pixels)
LC_PROP_G_CAMERA_HEIGHT	int	R	Height of camera image (pixels)
LC_PROP_G_CAMERA_BITS	handle int	R	Pointer to image bits
LC_PROP_G_CAMERA_BPROW	int	R	Bytes per row
Options of export a drawing to raster image (<u>lcBlockRasterize</u>)			
LC_PROP_G_RAS_FILL	bool	RW	Enable entities fillings
LC_PROP_G_RAS_COLOR	bool	RW	Enable entities colors
LC_PROP_G_RAS_NOPRINT	bool	RW	Apply layer "no print" option

Result valriables of the fucntion lcTIN_TriGetEdge

Graphics window [Home](#)

The LiteCAD graphics window is used to display and edit graphics data contained in the [drawing](#) object. Put it as a child-window it on the desired window/form of your application, by call the function [lcCreateWindow](#), and link it with a drawing by the function [lcWndSetBlock](#).

LiteCAD graphics window have service objects [Magnifier](#) and [Aerial View](#).

These are the LiteCAD window functions:

Function	Meaning
lcCreateWindow	Creates the window
lcDeleteWindow	Deletes the window
lcWndOnClose	Stops active processes before a window to be closed
lcWndExeCommand	Executes interactive command
lcWndResize	Changes the size and position of the window
lcWndRedraw	Redraws content of the window
lcWndRedrawAuto	Activates multiple redraw by timer
lcWndSetFocus	Sets the keyboard focus to the window
lcWndSetExtents	Sets drawing's extents for the window
lcWndSetBlock	Links graphics window with a block
lcWndSetProps	Links graphics window with a properties window
lcWndSetCmdwin	Links graphics window with a command window
lcWndSetBasePoint	Set base point for Polar tracking
lcWndEmulator	Pen movement emulation
lcWndMagnifier	Defines Magnifier feature
lcWndHoverText	Defines hover text
lcWndMessage	Displays message box
lcWndPickEnt	Let user to pick an entity on a window.
lcWndWaitPoint	Waits until user pick a point on a screen
lcWndWaitPoint2	Waits until user pick a point on a screen
lcWndInputStr	Input text string for a command
lcWndUpdate	Updates a window
lcWndDrwAdd	Adds a drawing
lcWndDrwDelete	Deletes a drawing
lcWndDrwGet	Enumerates all drawings of graphics window
Zoom	
lcWndZoomRect	Defines a part of a drawing, visible in the window
lcWndZoomScale	Changes visible scale of a drawing in the window (Zoom In / Zoom Out)
lcWndZoomMove	Offsets visible part of a drawing (Pan view)
lcWndZoomPos	Set view center and optional scale
lcWndZoomEnt	Zoom on specified entity

Function	Meaning
Coordinates	
<u>lcWndGetCursorCoord</u>	Returns cursor coordinates
<u>lcCoordDrwToWnd</u>	Converts a drawing coordinates into a window coordinates
<u>lcWndCoordFromDrw</u>	
<u>lcCoordWndToDrw</u>	Converts a window coordinates into a drawing coordinates
<u>lcWndCoordToDrw</u>	
Retrieve entities	
<u>lcWndGetEntByPoint</u>	Gets one entity at specified position (window coordinates)
<u>lcWndGetEntByPoint2</u>	Gets one entity at specified position (drawing coordinates)
<u>lcWndGetEntsByPoint</u>	Gets an array of entities at specified position (window coordinates)
<u>lcWndGetEntsByRect</u>	Gets an array of entities by specified rectangle (drawing coordinates)
<u>lcWndGetEntity</u>	Returns a handle to entity in the array
<u>lcWndGetEntByID</u>	Retrieves an entity by its identifier
<u>lcWndGetEntByIDH</u>	
<u>lcWndGetEntByKey</u>	Retrieves a graphic object by its key value

Paint mode functions

The window object has the following properties:

Property	Type	Access	Meaning
LC_PROP_WND_ID	int	RW	Window Identifier
LC_PROP_WND_WIDTH	int	R	Width of drawing's area, pixels
LC_PROP_WND_HEIGHT	int	R	Height of drawing's area, pixels
LC_PROP_WND_FRAME_LEFT	int	R	X position of the frame's left-top corner on parent window, pixels
LC_PROP_WND_FRAME_TOP	int	R	Y position of the frame's left-top corner on parent window, pixels
LC_PROP_WND_FRAME_WIDTH	int	R	Window frame width, pixels
LC_PROP_WND_FRAME_HEIGHT	int	R	Window frame height, pixels
LC_PROP_WND_HWND	handle	R	WinAPI handler (<u>HWND</u>) of a window
LC_PROP_WND_BLOCK	handle	R	Handle to a <u>block</u> linked with a window
LC_PROP_WND_VIEWBLOCK			
LC_PROP_WND_DRW	handle	R	Handle to a <u>drawing</u> currently displayed in a window
LC_PROP_WND_HASFOCUS	bool	R	TRUE, if a window have input focus
LC_PROP_WND_PIXELSIZE	float	RW	Current scale of the drawing in the window, units per pixel
LC_PROP_WND_SELECT	bool	RW	Enable / disable selection of entities

Property	Type	Access	Meaning
LC_PROP_WND_DTIME	int	R	Redraw time, milliseconds
LC_PROP_WND_FROZEN	bool	RW	"Frozen" mode
LC_PROP_WND_FROZENVIEW	bool	RW	"Frozen View" mode
LC_PROP_WND_COMMAND	int		Id of active command
LC_PROP_WND_CMD	handle	R	Handle to active command
	bool		TRUE if has active command
LC_PROP_WND_CMDENT1	bool	RW	TRUE - only one entity added by a command, FALSE - several entities
LC_PROP_WND_OSNAP	int		<u>Object snap</u> mode
	bool	RW	(LC_OSNAP_NODE and others) On/Off
LC_PROP_WND_OSNAPMENU	bool	RW	On/Off <u>Object snap</u> popup menu on <Shift> + <Right click>
LC_PROP_WND_PTRACK	bool	RW	Enable <u>Polar tracking</u>
LC_PROP_WND_PTRACK_ANGLE	float	RW	Step angle for <u>Polar tracking</u>
LC_PROP_WND_PTRACK_ANGREL	bool	RW	If true then LC_PROP_WND_PTRACK_ANGLE is relative, false - absolute
LC_PROP_WND_PTRACK_DIST	bool		Use distance step along polar vector
	float	RW	Value of distance step
LC_PROP_WND_BASEPT	bool	R	Has base point (Used for <u>Polar tracking</u>)
LC_PROP_WND_BASEX	float	R	X coordinate of base point
LC_PROP_WND_BASEY	float	R	Y coordinate of base point
LC_PROP_WND_ORTHO	bool	RW	Ortho mode
LC_PROP_WND_BGIMAGE	string		Filename of background image
	bool	RW	Display background image (if loaded)
LC_PROP_WND_TIN	handle	RW	Active <u>TIN object</u>
LC_PROP_WND_HASFILETABS	bool	R	TRUE if window has file tabs (was <u>created</u> with style LC_WS_FILETABS)
LC_PROP_WND_NUMFILETABS	int	R	Number of file tabs
LC_PROP_WND_NUMDRW	int	R	Number of drawings that graphics window contains
LC_PROP_WND_ENT	handle	R	Handle of picked entity. See <u>lcWndPickEnt</u>
LC_PROP_WND_DROPFILES	bool	RW	Registers whether a window accepts dropped files
LC_PROP_WND_ZOOMWHEEL	bool	RW	Enable zoom by mouse wheel

Visibility of some elements

Property	Type	Access	Meaning
LC_PROP_WND_RULERS	bool	RW	Display window rulers (The window must be created with LC_WS_RULERS style flag)
LC_PROP_WND_MAGNIFIER	bool	RW	Display <u>Magnifier</u> at right-bottom corner
LC_PROP_WND_NAVIGATOR	bool	RW	Display " <u>Aerial view</u> " window
LC_PROP_WND_BREAKPOINTS	bool	RW	Display breakpoints
LC_PROP_WND_BREAKPTNUMS	bool	RW	Display breakpoint number
LC_PROP_WND_JUMLINES	bool	RW	Display <u>jump lines</u>
LC_PROP_WND_ALPHABLEND	bool	RW	Enable alpha blend (transparent filling)
LC_PROP_WND_STDBLKFRAME	bool	RW	Draw red frame around a window if standard <u>block</u> is active
LC_PROP_WND_BLKBASEPT	bool	RW	Display <u>block</u> basepoint (LC_PROP_BLOCK_X, LC_PROP_BLOCK_Y)
LC_PROP_WND_DRAWPAPER	bool	RW	Draw paper sheet for "Paper space" blocks
LC_PROP_WND_LTVIEWMIN	int	RW	Linetype visibility, min size of pattern (pixels)
LC_PROP_WND_LTVIEWMAX	int	RW	Linetype visibility, max size of pattern (pixels)
Cursor type and position			
LC_PROP_WND_CURSORARROW LC_PROP_WND_CURSORSYS	bool	RW	Enable arrow cursor
	int	W	Set cursor by id (LC_CURSOR_ARROW, LC_CURSOR_CROSS, etc.)
	handle	RW	Get cursor handle / Set cursor by handle (HCURSOR)
LC_PROP_WND_CURSORCROSS	bool	RW	Enable crosshair cursor
LC_PROP_WND_CURSORSIZE	int	RW	Size of crosshair cursor, % of screen, if negative - size in pixels
LC_PROP_WND_CURSORPBOX	bool	RW	Enable a selecting square (pickbox) at cursor position
LC_PROP_WND_PICKBOXSIZE	float	R	Half size of a selecting square (drawing's units)
LC_PROP_WND_CURX	int	R	Cursor position X, pixels
	float		Cursor position X, drawing's units
LC_PROP_WND_CURY	int	R	Cursor position Y, pixels
	float		Cursor position Y, drawing's units
LC_PROP_WND_CURLEF	float	R	Left side coordinate of selecting square (drawing units)
LC_PROP_WND_CURBOT	float	R	Bottom side coordinate of selecting square (drawing units)

Property	Type	Access	Meaning
LC_PROP_WND_CURRIG	float	R	Right side coordinate of selecting square (drawing units)
LC_PROP_WND_CURTOP	float	R	Top side coordinate of selecting square (drawing units)
LC_PROP_WND_COORDS	bool	RW	Display cursor coordinates at left-bottom corner
Drawing view coordinates			
LC_PROP_WND_XMIN	float	R	Xmin of currently visible drawing's area
LC_PROP_WND_YMIN	float	R	Ymin of currently visible drawing's area
LC_PROP_WND_XMAX	float	R	Xmax of currently visible drawing's area
LC_PROP_WND_YMAX	float	R	Ymax of currently visible drawing's area
LC_PROP_WND_XCEN	float	R	Center X of currently visible drawing's area
LC_PROP_WND_YCEN	float	R	Center Y of currently visible drawing's area
LC_PROP_WND_DX	float	R	Width of currently visible drawing's area
LC_PROP_WND_DY	float	R	Height X of currently visible drawing's area
<u>Coordinate grid</u>			
LC_PROP_WND_GRIDSNAP	bool	RW	Enable cursor snap to grid nodes
LC_PROP_WND_GRIDSHOW	bool	RW	Display coordinate grid
LC_PROP_WND_GRIDDX	float	RW	Distance between vertical grid lines
LC_PROP_WND_GRIDDY	float	RW	Distance between horizontal grid lines
LC_PROP_WND_GRIDX0	float	RW	Grid origin point Y
LC_PROP_WND_GRIDY0	float	RW	Grid origin point X
LC_PROP_WND_GRIDBOLDX	int	RW	X step for grid bold lines
LC_PROP_WND_GRIDBOLDY	int	RW	Y step for grid bold lines
LC_PROP_WND_GRIDCOLOR	int	RW	Color of grid line
LC_PROP_WND_GRIDDOTTED	bool	RW	Grid line is dotted
LC_PROP_WND_GRIDCOLOR2	int	RW	Color of grid bold line
LC_PROP_WND_GRIDDOTTED2	bool	RW	Grid bold line is dotted
Snap to rectangle border			
LC_PROP_WND_RSNAPE	bool	RW	Enable snap to rectangle border

Property	Type	Access	Meaning
LC_PROP_WND_RSNAPSHOW	bool	RW	Display snap rectanle
LC_PROP_WND_RSNAPLEF	float	RW	X left
LC_PROP_WND_RSNAPBOT	float	RW	Y bottom
LC_PROP_WND_RSNAPRIG	float	RW	X right
LC_PROP_WND_RSNAPTOP	float	RW	Y top
LC_PROP_WND_RSNAPW	float	RW	Width (from X left)
LC_PROP_WND_RSNAPH	float	RW	Height (from Y bottom)
LC_PROP_WND_RSNAPCOLORM	int	RW	Rectangle filling color on Model space
LC_PROP_WND_RSNAPCOLORP	int	RW	Rectangle filling color on Paper space

Pan optimization

LC_PROP_WND_PANSTEP	int	RW	Pan: minimal step, pixels
LC_PROP_WND_PANLW	bool	RW	Pan: don't display linewidths
LC_PROP_WND_PANIMAGE	bool	RW	Pan: don't draw raster images
LC_PROP_WND_PANFILL	bool	RW	Pan: don't fill polygons
LC_PROP_WND_PANPIXSZ	bool	RW	Pan: reduce resolution

Measurements

LC_PROP_WND_MEASCOLORPNT	int	RW	Measurement point color (COLORREF)
LC_PROP_WND_MEASCOLORLINE	int	RW	Measurement line color (COLORREF)
LC_PROP_WND_MEASLINESIZE	int	RW	Measurement line size
LC_PROP_WND_MEASFONTSIZE	int	RW	Measurement font size
LC_PROP_WND_MEASFILLAREA	int	RW	Fill area polygons with hatch

Colors

LC_PROP_WND_COLORINFBG	int	RW	Info box: background color
LC_PROP_WND_COLORINFBOARD	int	RW	Info box: border color
LC_PROP_WND_COLORINFTEXT	int	RW	Info box: text color

Magnifier

LC_PROP_WND_COLORINFBG	int	RW	Info box: background color
------------------------	-----	----	----------------------------

Clipping rectangles

LC_PROP_WND_CRECTS_EDIT	bool	RW	"Edit Clipping Rectangles" mode
LC_PROP_WND_CRECTS_VISIBLE	bool	RW	Display clipping rectangles when LC_PROP_WND_CRECTS_EDIT is false

Move entities by keyboard

Property	Type	Access	Meaning
LC_PROP_WND_KBMOVE_ENABLE	bool	RW	Enable entities movement by keyboard
LC_PROP_WND_KBMOVE_DIST	float	RW	Distance step
LC_PROP_WND_KBMOVE_ANGLE	float	RW	Angle step
LC_PROP_WND_KBMOVE_SCALE	float	RW	Scale step
User specified data			
LC_PROP_WND_INT0			
...	int	RW	User integer variables
LC_PROP_WND_INT9			
LC_PROP_WND_FLOAT0			
...	float	RW	User float variables
LC_PROP_WND_FLOAT9			
LC_PROP_WND_STR0			
...	string	RW	User text variables
LC_PROP_WND_STR9			

If a drawing is not linked with a window, the following properties are available:

Property	Type	Access	Meaning
LC_PROP_WND_LWMODE	int	RW	Line width mode (LC_LW_THIN, LC_LW_REAL, LC_LW_PIXEL)
LC_PROP_WND_LWSCALE	float	RW	Screen scale for line width, mm/pixel (for LC_LW_PIXEL mode)
LC_PROP_WND_COLORBG	int	RW	Window background color
LC_PROP_WND_COLORCURSOR	int	RW	Cursor color
LC_PROP_WND_COLORFORE	int	RW	Foreground color

Events [Home](#)

Event processing permits you to control user actions while the user is running your application. LiteCAD features several types of events that can be used to instantiate additional procedural code. For each of these events you create an "event procedure" in your application. It has the following syntax:

```
void __stdcall EventProc (  
    HANDLE hEvent  
);
```

You have to define only those event procedures that you intend to use. Whenever the user performs any specified action, LiteCAD calls your previously defined corresponding procedure. Some events are manifested by setting a return value, in those cases, LiteCAD's behavior is dependant upon the return value.

These are service functions used to control events:

Function	Event
<u>lcEventSetProc</u>	Defines an application-defined event procedure for specific event
<u>lcEventReturnCode</u>	Defines return code of event procedure
<u>lcEventsEnable</u>	Enables or disables events generation

LiteCAD has the following types of events (defined by LC_PROP_EVENT_TYPE property):

Event type	Meaning
Mouse and keyboard	
<u>LC_EVENT_MOUSEMOVE</u>	Window cursor position has been changed
<u>LC_EVENT_LBDOWN</u>	Left mouse button has been pressed
<u>LC_EVENT_LBUP</u>	Left mouse button has been released
<u>LC_EVENT_LBDBLCLK</u>	Left mouse button double click
<u>LC_EVENT_RBDOWN</u>	Right mouse button has been pressed
<u>LC_EVENT_RBUP</u>	Right mouse button has been released
<u>LC_EVENT_SNAP</u>	Allows a mouse to stick to specific points
<u>LC_EVENT_KEYDOWN</u>	Keyboard key has been pressed
View	
<u>LC_EVENT_PAINT</u>	LiteCAD window is being redrawn
<u>LC_EVENT_WNDVIEW</u>	View position has been changed in LiteCAD window
<u>LC_EVENT_FILETAB</u>	User selects File Tab (to change active drawing in a window)
<u>LC_EVENT_VIEWBLOCK</u>	Window has changed active block

Entity

Event type	Meaning
<u>LC_EVENT_ADDENTITY</u>	Graphic entity has been added
<u>LC_EVENT_ENTPROP</u>	Entity's property value has been changed
<u>LC_EVENT_ENTMOVE</u>	Entity has been moved
<u>LC_EVENT_ENTSCALE</u>	Entity has been scaled
<u>LC_EVENT_ENTROTATE</u>	Entity has been rotated
<u>LC_EVENT_ENTMIRROR</u>	Entity has been mirrored
<u>LC_EVENT_ENTERASE</u>	Entity flag "Deleted" was changed
<u>LC_EVENT_GRIPMOVE</u>	User has moved entity's grip
<u>LC_EVENT_GRIPDRAG</u>	User is dragging entity's grip
<u>LC_EVENT_PICKENT</u>	User clicks on entity
<u>LC_EVENT_SELECT</u>	Selection set has been changed
<u>LC_EVENT_DRAWIMAGE</u>	Draw custom images
Drawing	
<u>LC_EVENT_DRWPROP</u>	Drawing's property value has been changed
<u>LC_EVENT_FILE</u>	Drawing has been loaded from a file or saved to a file
<u>LC_EVENT_EXTENTS</u>	Drawing's extents has been changed
Custom command	
<u>LC_EVENT_ADDCMD</u>	LiteCAD initializes commands. Use this event to add <u>custom commands</u> .
<u>LC_EVENT_CCMD</u>	<u>Custom command</u> interface
<u>LC_EVENT_WAITPOINT</u>	Generated on mouse move while the function <u>lcWndWaitPoint</u> is running
Other	
<u>LC_EVENT_ADDSTR</u>	LiteCAD initializes GUI strings
<u>LC_EVENT_HELP</u>	User clicks on "Help" button in LiteCAD dialog
<u>LC_EVENT_MAGNIFIER</u>	<u>Magnifier</u> property has been changed
<u>LC_EVENT_NAVIGATOR</u>	<u>Navigator</u> (aerial view) property has been changed
<u>LC_EVENT_GRID</u>	<u>Coordinate grid</u> property has been changed
<u>LC_EVENT_OSNAP</u>	<u>Object snap</u> property has been changed
<u>LC_EVENT_PTRACK</u>	<u>Polar tracking</u> property has been changed
<u>LC_EVENT_ORTHO</u>	"Ortho mode" property has been changed

Some events are manifested by setting a return value, in those cases, LiteCAD's behavior is dependant upon the return value.

The function **lcEventReturnCode** is used to set event return code.

Inside the event procedure, you can access the following event properties by using the **hEvent** parameter:

Property	Type	Access	Meaning
LC_PROP_EVENT_TYPE	int	R	Event type (LC_EVENT_MOUSEMOVE and other)
LC_PROP_EVENT_APPPRM1	int	R	Application-defined parameter 1
LC_PROP_EVENT_APPPRM2	handle	R	Application-defined parameter 2
LC_PROP_EVENT_WND	handle	R	<u>LiteCAD design window</u>
LC_PROP_EVENT_DRW	handle	R	<u>Drawing</u>
LC_PROP_EVENT_BLOCK	handle	R	<u>Block</u>
LC_PROP_EVENT_ENTITY	handle	R	<u>Entity</u>
LC_PROP_EVENT_HDC	handle	R	Device context
LC_PROP_EVENT_HCMD	handle	R	Object defined in a custom command
LC_PROP_EVENT_INT1	int	RW	Integer value. The meaning depends on event type.
LC_PROP_EVENT_INT2			
LC_PROP_EVENT_INT3			
LC_PROP_EVENT_INT4			
LC_PROP_EVENT_INT5			
LC_PROP_EVENT_FLOAT1	float	RW	Float value. The meaning depends on event type.
LC_PROP_EVENT_FLOAT2			
LC_PROP_EVENT_FLOAT3			
LC_PROP_EVENT_FLOAT4			
LC_PROP_EVENT_FLOAT5			
LC_PROP_EVENT_FLOAT6			
LC_PROP_EVENT_STR1	string	RW	string value. The meaning depends on event type.
LC_PROP_EVENT_STR2			
LC_PROP_EVENT_STR3			

Code sample:

```
// LiteCAD event procedure
//-----
void CALLBACK EventProc (HANDLE hEvent)
{
    int EventType;
    LcApplication* pApp = (LcApplication*)lcPropGetHandle( hEvent, LC_PROP_EVENT_APPPRM2 );
    if (pApp){
        EventType = lcPropGetInt( hEvent, LC_PROP_EVENT_TYPE );
        switch( EventType ){
            case LC_EVENT_MOUSEMOVE:  pApp->OnMouseMove( hEvent ); break;
            case LC_EVENT_KEYDOWN:    pApp->OnKeyDown( hEvent ); break;
            ...
        }
    }
}

//-----
bool LcApplication::Init (HINSTANCE hInst, LPCWSTR szAppDir)
{
    BOOL bInit;
    ...
    lcEventSetProc( LC_EVENT_MOUSEMOVE, EventProc, 0, (HANDLE)this );
    lcEventSetProc( LC_EVENT_KEYDOWN, EventProc, 0, (HANDLE)this );
}
```

```

...
bInit = lcInitialize();
...
}

//-----
void LcApplication::OnMouseMove (HANDLE hEvent)
{
    WCHAR szBuf[256], szNum[16];
    double X, Y;
    HANDLE hLcWnd = lcPropGetHandle( hEvent, LC_PROP_EVENT_WND );

    X = lcPropGetFloat( hLcWnd, LC_PROP_WND_CURX );
    Y = lcPropGetFloat( hLcWnd, LC_PROP_WND_CURY );

    // X
    okDblToStr( X, szNum );
    swprintf( szBuf, L"X: %s", szNum );
    lcStatbarText( m_hStatBar, 1, szBuf );
    // Y
    okDblToStr( Y, szNum );
    swprintf( szBuf, L"Y: %s", szNum );
    lcStatbarText( m_hStatBar, 2, szBuf );

    lcStatbarRedraw( m_hStatBar );
}

//-----
void LcApplication::OnKeyDown (HANDLE hEvent)
{
    UINT VirtKey, Flags;
    BOOL bShift, bCtrl;
    double X, Y;
    HANDLE hLcWnd, hLcDrw, hBlock;

    VirtKey = lcPropGetInt( hEvent, LC_PROP_EVENT_INT1 );
    Flags   = lcPropGetInt( hEvent, LC_PROP_EVENT_INT2 );
    bShift   = lcPropGetInt( hEvent, LC_PROP_EVENT_INT3 );
    bCtrl    = lcPropGetInt( hEvent, LC_PROP_EVENT_INT4 );
    X = lcPropGetFloat( hEvent, LC_PROP_EVENT_FLOAT1 );
    Y = lcPropGetFloat( hEvent, LC_PROP_EVENT_FLOAT2 );
    hLcWnd = lcPropGetHandle( hEvent, LC_PROP_EVENT_WND );
    hLcDrw = lcPropGetHandle( hEvent, LC_PROP_EVENT_DRW );
    hBlock = lcPropGetHandle( hEvent, LC_PROP_EVENT_BLOCK );

    if (OnKeyDown( VirtKey, Flags )){
        lcEventReturnCode( 1 );
    }
    ...
}

```

Points buffer [Home](#)

Points buffer is a list of points that represent geometric shape. It can be drawn in a [window](#) with the function [lcPaint_DrawPtbuf](#) .

These are the functions used to manage points buffer:

Function	Meaning
lcPaint_CreatePtbuf	Creates point buffer object
lcPaint_DeletePtbuf	Deletes points buffer object
lcPaint_PtbufClear	Removes all points
lcPaint_PtbufAddPoint	Adds one point
lcPaint_PtbufAddPoint2	Adds one point
lcPaint_PtbufAddPointP	Adds one point, using polar offset from previous point
lcPaint_PtbufAddLine	Adds two points
lcPaint_PtbufAddLineP	Adds two points, using polar offset for second point
lcPaint_PtbufAddCircle	Adds a circle points (Center, Radius)
lcPaint_PtbufAddCircle2	Adds a circle points (2 opposite points)
lcPaint_PtbufAddCircle3	Adds a circle points (3 points)
lcPaint_PtbufAddArc	Adds an arc points (Center, Radius, start Angle, arc Angle)
lcPaint_PtbufAddArc3p	Adds an arc points (Start, Middle, End)
lcPaint_PtbufAddArcSDE	Adds an arc points (Start, Direction angle, End)
lcPaint_PtbufAddArcSDAR	Adds an arc points (Start, Direction angle, Arc angle, End)
lcPaint_PtbufAddArcSER	Adds an arc points (Start, End, Radius)
lcPaint_PtbufAddArcSEL	Adds an arc points (Start, End, arc Length)
lcPaint_PtbufAddArcSEA	Adds an arc points (Start, End, Arc angle)
lcPaint_PtbufAddArcSEB	Adds an arc points (Start, End, Bulge)
lcPaint_PtbufAddArcCSE	Adds an arc points (Center, Start, End)
lcPaint_PtbufAddArcCSA	Adds an arc points (Center, Start, Arc angle)
lcPaint_PtbufAddArcCSL	Adds an arc points (Center, Start, chord Length)
lcPaint_PtbufAddArcCRAA	Adds an arc points (Center, Radius, start Angle, end Angle)
lcPaint_PtbufAddEllipse	Adds an ellipse or elliptical arc
lcPaint_PtbufAddEllipse2	Adds an ellipse, inscribed into a rectangle
lcPaint_PtbufAddRect	Adds a rectangle (Center, Width, Height, Rotation)
lcPaint_PtbufAddRect2	Adds a rectangle (2 points, no rotation)
lcPaint_PtbufAddRect3	Adds a rectangle (3 points)
lcPaint_PtbufAddWline	Adds a wide line polygon
lcPaint_PtbufAddPtbuf	Adds points from other buffer
lcPaint_PtbufGetPoint	Retrieves a point coordinates
lcPaint_PtbufGetPoint2	
lcPaint_PtbufInterpolate	Generates new set of points as a result of source points interpolation

Function	Meaning
<u>lcPaint_PtbufMove</u>	Moves all points
<u>lcPaint_PtbufRotate</u>	Rotates all points
<u>lcPaint_PtbufScale</u>	Scales all points
<u>lcPaint_PtbufMirror</u>	Mirrors all points
<u>lcPaint_PtbufCopy</u>	Copies all points into other points buffer

Multipolygon [Home](#)

Multipolygon object is a list of polygons. It is used to display filled shapes. It can be drawn in a [window](#) with the function [lcPaint_DrawMpgon](#) .

These are the functions used to manage multipolygon object:

Function	Meaning
lcPaint_CreateMpgon	Creates multipolygon object
lcPaint_DeleteMpgon	Deletes multipolygon object
lcPaint_MpgonClear	Removes all polygons
lcPaint_MpgonAddPgon	Adds new polygon from points buffer
lcPaint_MpgonAddText	Adds text polygons
lcPaint_MpgonAddBarcode	Adds barcode polygons
lcPaint_MpgonMove	Moves multipolygon
lcPaint_MpgonRotate	Rotates multipolygon
lcPaint_MpgonScale	Scales multipolygon
lcPaint_MpgonMirror	Mirrors multipolygon
lcPaint_MpgonCopy	Copies multipolygon

This are [properties](#) of a multipolygon object:

Property	Type	Access	Meaning
LC_PROP_MPGON_XMIN	float	R	X min
LC_PROP_MPGON_YMIN	float	R	Y min
LC_PROP_MPGON_XMAX	float	R	X max
LC_PROP_MPGON_YMAX	float	R	Y max
LC_PROP_MPGON_XCEN	float	R	X of extents center
LC_PROP_MPGON_YCEN	float	R	Y of extents center
LC_PROP_MPGON_W	float	R	Width of extents rect
LC_PROP_MPGON_H	float	R	Height of extents rect

Raster Image [Home](#)

LiteCAD stores all raster images in a global list. This are functions used to add/remove objects in the list:

Function	Meaning
<u>lcPaint_ImageAdd</u>	Adds new image object into images list
<u>lcPaint_ImageDelete</u>	Deletes image object from images list
<u>lcPaint_ImageGetFirst</u>	Used to sequentially retrieve all image objects from a list
<u>lcPaint_ImageGetNext</u>	
<u>lcPaint_ImageGetByID</u>	Retrieves an image object by its identifier

Raster image can be drawn in a [window](#) with the function [lcPaint_DrawImage](#) .

These are the functions used to manage raster image:

Function	Meaning
<u>lcPaint_ImageLoad</u>	Loads image from a file
<u>lcPaint_ImageCopy</u>	Copies image from other object
<u>lcPaint_ImageCreate</u>	Creates empty image
<u>lcPaint_ImageSetPixel</u>	Sets pixel color
<u>lcPaint_ImageFlip</u>	Flips image horizontally or/and vertically
<u>lcPaint_ImageRotate</u>	Rotates image
<u>lcPaint_ImageGray</u>	Converts color image to grayscale
<u>lcPaint_ImageResize</u>	Upsampling / downsampling
<u>lcPaint_ImageGetPtbuf</u>	Creates clipping polygon for rotated image

Raster image has the following [properties](#):

Property	Type	Access	Meaning
LC_PROP_IMAGE_ID	int	R	Identifier
LC_PROP_IMAGE_FILENAME	string	R	Filename
LC_PROP_IMAGE_W	int	R	Width, pixels
LC_PROP_IMAGE_H	int	R	Height, pixels

Text Font [Home](#)

During initialization ([lcInitialize](#)), LiteCAD builds a list of available fonts by scanning Windows system fonts (TrueType) and own fonts (LCF) in the directory specified by [LC_PROP_G_DIRFONTS](#) [property](#).

In order to use a font to [draw text](#) in a [window](#), you have to open desired font and select it.

These are the functions used to manage text fonts:

Function	Meaning
lcFontAddRes	
lcFontAddFile	Adds a font to the list of available fonts, from various sources
lcFontAddBin	
lcFontGetFirst	
lcFontGetNext	Used to sequentially retrieve objects from a list of available fonts

For Paint mode

lcPaint_FontOpenLC	Opens LiteCAD font
lcPaint_FontOpenTT	Opens TrueType font
lcPaint_FontClose	Closes opened font
lcPaint_FontSelect	Select opened font, which will be used to draw texts

A font object has the following [properties](#):

Property	Type	Access	Meaning
LC_PROP_FONT_FILENAME	string	R	Font filename
LC_PROP_FONT_NAME	string	R	Font name
LC_PROP_FONT_LCF	bool	R	TRUE - LCF format, FALSE - TTF format
LC_PROP_FONT_TTF	bool	R	TRUE - TTF format, FALSE - LCF format
LC_PROP_FONT_HEIGHT	float	R	Base height
LC_PROP_FONT_FILLED	bool	R	If TRUE then characters have closed outline, i.e. can be filled
LC_PROP_FONT_NCHARS	int	R	Number of characters in the font

Localization [Home](#)

These are the functions used for localization of GUI strings:

Function	Meaning
<u>lcStrAdd</u>	Adds text string for application
<u>lcStrSet</u>	Replaces text string
<u>lcStrGet</u>	Returns text string
<u>lcStrFileLoad</u>	Loads text strings from language file
<u>lcStrFileSave</u>	Save default text strings into language file

The above functions, except **lcStrGet**, are intended to use inside of the LC_EVENT_ADDSTR event procedure.

Localization of LiteCAD.exe

- Run **LiteCAD.exe** and press <Tab> key - command window will appear.
- In the command window enter "savestr". This command will create file **LC_original.lng** in the **Config** directory. File **LC_original.lng** contains original GUI strings of LiteCAD.
- Run application **EditLng.exe**, which is used to create LNG file with translated strings.
- By menu "File / Open Original..." load the file **LC_original.lng**
- Edit fields in the "String" column, with required language. After finishing - save contents of this column in LNG file, for example **LC_russian.lng** . The file will be placed in the **Config** directory.
- Open the file **LiteCAD_exe.cfg** in text editor and find the string LANGUAGE=
- Add name of the created LNG file, for example LANGUAGE=LC_russian.lng
- Save and close the file **LiteCAD_exe.cfg**
- Run **LiteCAD.exe** . Now all replaced GUI strings will be loaded from **LC_russian.lng** . For not replaced strings, LiteCAD will use original.

In your own application you can load LNG file in the LC_EVENT_ADDSTR event procedure by the function lcStrFileLoad , for example:

```
lcStrFileLoad( L"LC_russian.lng" );
```

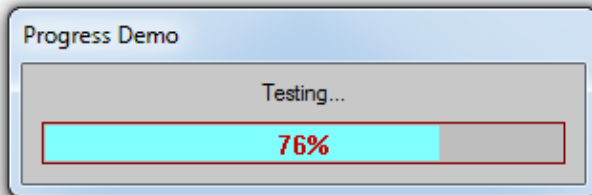
Progress indicator [Home](#)

These are the progress indicator functions:

Function	Meaning
<u>lcProgressCreate</u>	Creates progress indicator window
<u>lcProgressSetText</u>	Sets a text displayed inside of the window.
<u>lcProgressStart</u>	Sets a range of progress positions
<u>lcProgressSetPos</u>	Sets progress bar position
<u>lcProgressInc</u>	Increments progress bar position
<u>lcProgressDelete</u>	Deletes progress indicator window

Code sample:

```
//-----
void LcAppDemo01::TestProgress ()
{
    int MaxVal = 5000000;
    int i;
    if (lcProgressCreate( m_hwMain, 300, 90, L"Progress Demo" )){
        lcProgressSetText( L"Testing..." );
        lcProgressStart( 0, MaxVal );
        for (i=0; i<MaxVal; i++){
            ...
            lcProgressInc();
        }
        lcProgressDelete();
        lcWndSetFocus( m_hLcWnd );
    }
}
```



Quadrilateral [Home](#)

These are functions used to convert coordinates between quadrilaterals:

Function	Meaning
<u>lcQuadCreate</u>	Creates quadrilateral object
<u>lcQuadDelete</u>	Deletes quadrilateral object
<u>lcQuadSet</u>	Defines vertices of quadrilateral
<u>lcQuadTransXYtoUV</u>	Converts XY coordinates into UV coordinates
<u>lcQuadTransUVtoXY</u>	Converts UV coordinates into XY coordinates
<u>lcQuadContains</u>	Determines whether quadrilateral contains a specified point

Miscellaneous functions [Home](#)

These are the functions:

Function	Meaning
<u>lcPrintSetup</u>	Calls "Printer Setup" dialog
<u>lcPrintLayout</u>	Prints "Paper space" block, entire page
<u>lcPrintBlock</u>	Prints any rectangular area of a block
<u>lcDgGetValue</u>	Calls dialog box to input a value
<u>lcHelp</u>	Calls help file
<u>lcGetPolarPoint</u>	Retrieves point coordinates by angle and distance
<u>lcGetPolarPrm</u>	Retrieves a distance and angle between 2 points
<u>lcFilletSetLines</u>	
<u>lcFillet</u>	Calculate points of fillet arc
<u>lcFilletGetPoint</u>	
<u>lcPlugGetOption</u>	Returns a value of plugin's option
<u>lcPlugGetOption2</u>	
<u>lcPlugSetOption</u>	Sets a value of plugin's option
<u>lcGetDrwXData</u>	Retrieves drawing's XDATA from a file without loading entire file.
<u>lcGetDrwPreview</u>	Retrieves drawing's preview image from a file without loading entire file.

Progress indicator

Clipping rectangles